Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251211**
Nama Lengkap	**Jendri Mansamnaf Japen**
Minggu ke / Materi	**07 / Percabangan dan Perulangan Kompleks**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

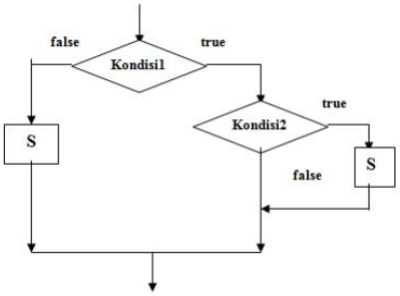
BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 6.3.1 Struktur Percabangan Komplek

Percabangan Kompleks Bentuk 1

Percabangan kompleks adalah struktur kontrol dalam pemrograman yang digunakan ketika suatu kondisi tidak hanya memiliki satu pilihan, tetapi terdiri dari beberapa alternatif. Dalam percabangan ini, program akan mengecek lebih dari satu kondisi secara berurutan.



Gambar 6.1: Flowchart Percabangan Kompleks Bentuk 1

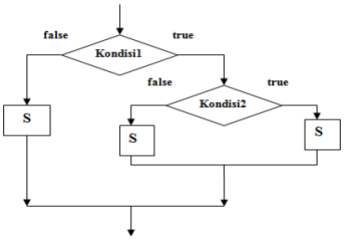
Percabangan Kompleks Bentuk 2

Percabangan kompleks bentuk 2 merupakan percabangan bersarang (*nested if*), yaitu if di dalam if. Pada struktur ini, kondisi kedua hanya akan diperiksa jika kondisi pertama terpenuhi.

Alurnya sebagai berikut:

1. Program pertama-tama mengecek kondisi1.
2. Jika kondisi1 benar, maka program masuk ke dalam blok tersebut dan akan mengecek kondisi2.
    a. Jika kondisi2 benar, maka perintah pada bagian itu dijalankan.
    b. Jika kondisi2 salah, maka bagian else dari kondisi2 yang dijalankan.
3. Jika kondisi1 salah, maka program langsung menjalankan bagian else dari kondisi1 tanpa mengecek kondisi2.

Jadi, pada bentuk ini kondisi kedua bergantung pada kondisi pertama. Artinya, kondisi2 tidak akan pernah diperiksa jika kondisi1 tidak terpenuhi. Kesimpulannya, percabangan ini digunakan ketika ada kondisi lanjutan yang hanya relevan jika kondisi utama sudah benar. Struktur ini sering dipakai untuk proses seleksi bertingkat atau keputusan yang saling bergantung.



Gambar 6.2: Flowchart Percabangan Kompleks Bentuk 2

Percabangan Kompleks Bentuk 3

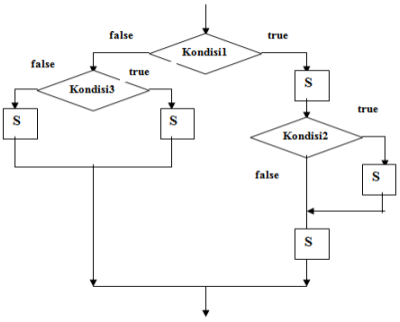
Pada bentuk 3, percabangan memiliki kombinasi antara kondisi yang berdiri sendiri dan kondisi bersarang. Setelah kondisi pertama dicek dan dijalankan jika benar, program akan melanjutkan ke kondisi berikutnya.

Jika kondisi2 tidak terpenuhi, maka program tidak langsung berhenti, tetapi masuk ke bagian else yang berisi kondisi3 sebagai alternatif lanjutan. Artinya, kondisi3 hanya akan diperiksa jika kondisi2 salah.

Alurnya:

- Cek kondisi1 => jika benar, jalankan perintah.
- Cek kondisi2:
    - Jika benar => jalankan perintah.
    - Jika salah => cek kondisi3:
        - Jika benar => jalankan perintah.

Kesimpulannya, bentuk ini digunakan ketika ada kondisi cadangan (backup) jika kondisi sebelumnya tidak terpenuhi. Jadi, alur keputusan menjadi lebih fleksibel dan bertingkat.



Gambar 6.3: Flowchart Percabangan Kompleks Bentuk 3

Percabangan Kompleks Bentuk 4

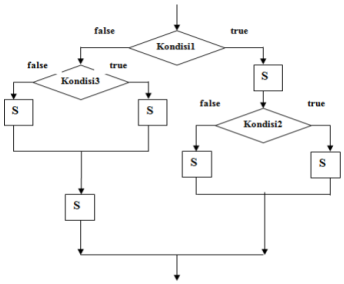
Percabangan kompleks bentuk 4 merupakan struktur yang lebih lengkap karena menggabungkan percabangan bersarang dan percabangan alternatif dalam dua bagian utama.

Alurnya:

1. Program mengecek kondisi1 terlebih dahulu.
    a. Jika kondisi1 benar:
        1. Jalankan perintah.
        2. Lalu cek kondisi2:
            1. Jika kondisi2 benar, jalankan perintah.
            2. Jika kondisi2 salah, jalankan bagian else dari kondisi2.
    b. Jika kondisi1 salah:
        1. Program masuk ke bagian else dari kondisi1.
        2. Di dalamnya terdapat kondisi3:
            1. Jika kondisi3 benar, jalankan perintah.
            2. Jika kondisi3 salah, jalankan bagian else dari kondisi3.

Jadi, struktur ini memiliki dua cabang besar:

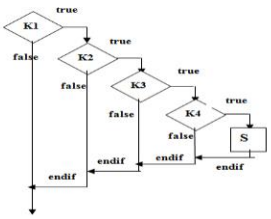
1. Cabang saat kondisi1 benar => memiliki percabangan lagi (kondisi2)
2. Cabang saat kondisi1 salah => memiliki percabangan alternatif (kondisi3)



Gambar 6.4: Flowchart Percabangan Kompleks Bentuk 4

Percabangan kompleks Bentuk 5

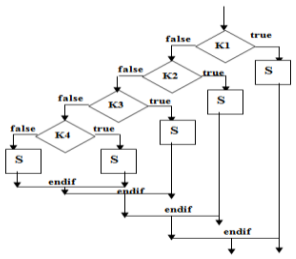
Pada penggunaan IF biasa (if yang berdiri sendiri), setiap kondisi akan diperiksa satu per satu tanpa memperhatikan apakah kondisi sebelumnya sudah terpenuhi atau belum. Artinya, walaupun salah satu kondisi sudah bernilai benar dan perintahnya sudah dijalankan, program tetap akan melanjutkan pengecekan ke kondisi-kondisi berikutnya. Hal ini menyebabkan proses eksekusi menjadi kurang efisien, terutama jika jumlah kondisi cukup banyak, karena semua kondisi tetap diuji.



Gambar 6.5: Flowchart Percabangan Kompleks Bentuk 5

Percabangan Kompleks Bentuk 6

Percabangan dalam flowchart memiliki beberapa bentuk yang penggunaannya harus disesuaikan dengan kebutuhan logika program. Dua bentuk yang sering digunakan adalah if–elif–else (IF bertingkat) dan if biasa (if berdiri sendiri). Keduanya memiliki perbedaan mendasar dalam cara kerja dan tujuan penggunaannya. Pada if–elif–else, kondisi diperiksa secara berurutan dari atas ke bawah, dan ketika satu kondisi sudah terpenuhi, maka proses pengecekan langsung berhenti. Hal ini membuat hanya satu blok perintah yang dijalankan. Struktur ini sangat cocok digunakan dalam kasus di mana hasil yang diinginkan hanya satu, seperti pada konversi nilai angka ke huruf, penentuan kategori, atau pemilihan satu opsi dari beberapa pilihan. Selain itu, penggunaan if–elif–else membuat program lebih efisien karena tidak perlu memeriksa semua kondisi, serta lebih terstruktur dan mudah dipahami.

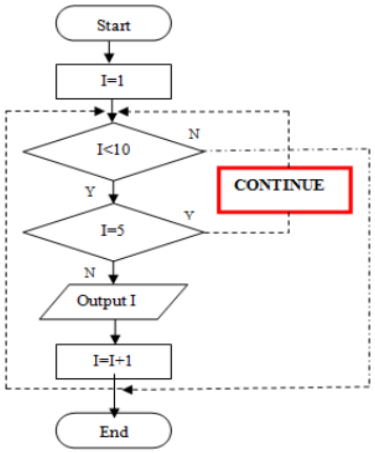


Gambar 6.6: Flowchart Percabangan Kompleks Bentuk 6

MATERI 6.3.2 Struktur Perulangan Kompleks Break

Perulangan Bertingkat

Perulangan bertingkat adalah perulangan di dalam perulangan (nested loop), di mana terdapat outer loop (perulangan luar) dan inner loop (perulangan dalam). Setiap satu kali outer loop dijalankan, inner loop akan berjalan berulang kali, sehingga total proses menjadi lebih banyak dan waktu eksekusi lebih lama.



Gambar 6.9: Flowchart Program Menggunakan Continue

Struktur ini sangat cocok digunakan untuk menyelesaikan masalah yang memiliki pola grid atau dua dimensi, seperti matriks, game board (catur, minesweeper), dan pengolahan citra. Meskipun lebih kompleks, perulangan bertingkat sangat berguna untuk menangani data yang berlapis atau berulang dalam dua arah (panjang dan lebar).

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Latihan 6.1

```
[1]: #Latihan 6.1

def is_prima(x):
    if x < 2:
        return False
    for i in range(2, int(x**0.5) + 1):
        if x % i == 0:
            return False
    return True

n = int(input("Masukan nilai n: "))

for i in range(n-1, 1, -1):
    if is_prima(i):
        print("Bilangan prima terdekat < n adalah: ", i)
        break
```

```
Masukan nilai n:  12
Bilangan prima terdekat < n adalah:  11
```

Program ini berhasil menemukan bilangan prima terbesar yang lebih kecil dari nilai input. Dengan input 12, program memeriksa angka-angka mundur dan menemukan 11 sebagai bilangan prima terdekat. Program berhenti setelah menemukan bilangan prima tersebut, sehingga proses pencarian menjadi efisien.

Latihan 6.2

```
[4]: #Latihan 6.2

     import math

     n = int(input("Masukan nilai n: "))

     for i in range(n, 0, -1):
         faktorial = math.factorial(i)
         print(f"{faktorial}", end=" ")
         for j in range(i, 0, -1):
             print(j, end=" ")
         print()
```

```
Masukan nilai n:  6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

Program ini menampilkan deret faktorial diikuti angka menurun dari n ke 1 secara dinamis berdasarkan input pengguna. Setiap baris pertama menampilkan faktorial dari angka saat itu, diikuti deret angka dari nilai tersebut ke 1. Looping bertingkat digunakan untuk menghitung faktorial dan menampilkan deret, sehingga menghasilkan pola yang diinginkan secara terstruktur.

Latihan 6.3

```
[5]: #Latihan 6.3

tinggi = int(input("Masukan tinggi: "))
lebar = int(input("Masukan lebar: "))

angka = 1
for i in range(tinggi):
    for j in range(lebar):
        print(angka, end=" ")
        angka += 1
    print()
```

```
Masukan tinggi:  5
Masukan lebar:  4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
```

Program ini menampilkan deret angka secara berturut-turut dalam bentuk baris dan kolom berdasarkan input tinggi dan lebar. Looping bertingkat digunakan untuk mengatur baris dan kolom, sedangkan variabel penghitung memastikan angka bertambah secara berurutan. Dengan cara ini, deret angka dapat dicetak secara rapi sesuai ukuran yang diinginkan.

- Link Github: https://github.com/japenjendri-design/Praktikkum-Algoritma-dan-Pemrograman.git